

Tolerância a Falhas

Relatório Trabalho Prático

Diogo Marques
PG55931

Francisco Ferreira
PG55942

Ivan Ribeiro
PG55950

Introdução

Neste trabalho prático, pretendemos identificar e analisar possíveis vulnerabilidades do algoritmo de *Raft* perante falhas assertivas que comprometam as propriedades de segurança do mesmo.

Inicialmente, descrevemos a nossa implementação do *Raft* e a sua integração com o ambiente *Maelstrom*, de seguida o uso do [Stateright](#) para verificar a correção do algoritmo face a possíveis falhas e respetivas mitigações.

Implementação do Raft

O algoritmo de Raft foi implementado na linguagem de programação *Rust*. Quando o programa é executado, é inicializado um nó *Maelstrom* que aguarda mensagens do *stdin* e escreve mensagens para o *stdout*.

A estrutura principal do código está dividida em módulos que separam as responsabilidades:

- **raft.rs**: Contém a lógica central do algoritmo de *Raft*, incluindo a máquina de estados dos nós (Seguidor, Candidato, Líder), a replicação de *logs* e os processos de eleição.
- **maelstrom.rs**: Providencia a abstração para interagir com o ambiente *Maelstrom*, tratando da serialização/desserialização de mensagens e da comunicação entre nós (ler do *stdin* e escrever para o *stdout*).
- **transport.rs**: Define uma interface de transporte para o *Raft* (*RaftTransport*) e uma implementação específica para *Maelstrom* (*MaelstromTransport*), desacoplando a lógica do mecanismo de transporte subjacente.
- **main.rs**: Inicializa o nó *Maelstrom*, integrando o sistema com os testes *lin-kv* do *Maelstrom*.

Para testar a implementação via *Maelstrom* foi utilizado o seguinte comando:

```
cargo build --release
java -jar maelstrom.jar test
  -w lin-kv
  --bin ".\target\release\pessi-raft.exe"
  --time-limit 60
  --node-count 3
  --concurrency 10n
  --rate 100
  --nemesis partition
  --nemesis-interval 3
```

Como resultado, observamos que o *Maelstrom* não detetou falhas no nosso algoritmo.

```
> Everything looks good!
```

No entanto, não é certo que o algoritmo esteja totalmente correto, pois o *Maelstrom* apenas testa um subconjunto limitado de cenários e não garante a cobertura de todos os estados possíveis ou falhas do sistema.

Verificação de Estados com o [Stateright](#)

Para além dos testes de integração com o *Maelstrom*, recorreu-se ao [Stateright](#) para uma verificação mais formal do algoritmo de *Raft* implementado. O [Stateright](#) é uma ferramenta de *model checking* para sistemas distribuídos escritos em *Rust*, que permite explorar exaustivamente os possíveis estados de um sistema e verificar se certas propriedades se mantêm, tudo a partir de código.

Funcionamento e Aplicação ao Raft

O *model checking* com [Stateright](#) envolve a definição de:

1. **Um modelo do sistema:** Este modelo descreve os estados possíveis de cada nó (e.g., `current_term`, `voted_for`, `log`, `commit_index`, `role`) e as ações que podem transitar o sistema de um estado para outro (e.g., enviar/receber uma mensagem `RequestVote`, `AppendEntries`, dar *timeout* numa eleição);
2. **Um ambiente:** Define como as ações dos nós interagem, incluindo a rede (que pode perder, duplicar ou reordenar mensagens) e outros eventos não determinísticos (mensagens que podem ser enviadas em qualquer momento);
3. **Propriedades:** São invariantes ou condições que devem ser sempre verdadeiras em todos os estados alcançáveis do sistema.

Esta definição foi realizada através de:

- Implementação do *Raft*, como modelo;
- Ambiente com uma rede que não perde, duplica ou reordena mensagens;
- Uma única mensagem, que consiste em escrever o valor 42 na chave 1;¹
- Uma lista de propriedades que iremos apresentar a seguir;
- Resto dependente do teste a fazer.

Propriedades Verificadas

Foram definidas e verificadas propriedades essenciais do Raft para garantir a sua correção:

1. **Segurança da Eleição (Election Safety):** No máximo um líder pode ser eleito num determinado termo.
 - Expectativa temporal²: Always
2. **Coerência do Log (Log Safety):** Se dois *logs* contêm uma entrada *committed* com o mesmo índice e termo, então os *logs* são idênticos até esse índice.
 - Expectativa temporal: Always
3. **Vivacidade do Log (Log Liveness):** O *log* deverá progredir, isto é, haver um líder que aplique entradas no *log*. O *log* não deverá ficar vazio eternamente.
 - Expectativa temporal: Sometimes

¹Esta mensagem foi escolhida de forma arbitrária e única para simplificar e acelerar a exploração dos possíveis estados do sistema, sendo suficiente para verificar todas as propriedades.

²Always significa que a propriedade deve ser respeitada em todos os estados alcançáveis do sistema, enquanto que Sometimes significa que a propriedade deve ser respeitada em alguns estados alcançáveis do sistema.

4. **Vivacidade de Eleições (Election Liveness):** O sistema deverá progredir de modo a que um líder seja eleito. Um líder deverá ser eleito inevitavelmente.
- Expectativa temporal: Sometimes

Estas propriedades, que se encontram em `src/fault/property.rs`, foram afortunadamente verificadas com o [Stateright](#) para o algoritmo de *Raft* implementado.

A implementação do nosso algoritmo de *Raft* é feita sobre uma abstração de transporte, o que nos permite facilmente criar um transporte específico para o [Stateright](#) sem modificar o código do algoritmo. Assim, conseguimos executar o *Raft* diretamente sobre este ambiente, facilitando a verificação formal das respetivas propriedades.

Faltas

Para analisar o comportamento do *Raft* perante falhas bizantinas e comportamentos maliciosos, criámos uma “API de Faltas” que permite injetar código em pontos críticos da execução. Pontos críticos são por exemplo, numa receção de mensagem, numa transição de estado importante (exemplo: líder eleito), etc. Desta forma, conseguimos simular anomalias sem alterar o código do algoritmo diretamente.

O impacto de cada falha foi avaliado a partir de testes unitários (usando `cargo test`) com a integração do [Stateright](#), permitindo verificar se as propriedades eram violadas sob essas condições, e se a mitigação de cada falha era eficaz.

Nas secções seguintes, detalhamos as faltas investigadas, o seu impacto potencial no sistema, a evidência de ocorrências, as possíveis medidas de mitigação e demonstração das mesmas.

Falta A - Falsificação/Adulteração de Mensagens

Identificação da Falta

Um nó malicioso pode falsificar ou adulterar mensagens de outros nós.

Impacto

O impacto desta falha é extremamente grave, pois um nó malicioso com capacidade de falsificar ou adulterar mensagens pode comprometer completamente a segurança e a confiabilidade do *cluster*. Entre os impactos possíveis, destacam-se:

- Derrubar eleições legítimas, falsificando mensagens de votação para impedir a eleição de um líder correto;
- Eleger-se a si próprio, ao manipular votos ou resultados de eleições;
- Injetar entradas falsas no *log*, comprometendo a integridade dos dados replicados;
- Causar divisões no *cluster*, enviando mensagens contraditórias para diferentes nós, levando a estados inconsistentes;
- Impedir a propagação de comandos legítimos, bloqueando ou adulterando mensagens de confirmação;

Em suma, a falsificação de mensagens permite a um atacante assumir controlo total do sistema, violando todas as propriedades de segurança e disponibilidade do protocolo *Raft*.

Demonstração do Impacto

A demonstração de falsificação pode ser feita alterando os campos onde é enviado o `node_id` da mensagem. A identificação do nó é feita em campos de mensagens do *Raft*, e estes não são validados.

```
struct VoteRequestMessage {  
    node_id: Id, /// <--- Identificação do nó que envia a mensagem  
    current_term: TermId,  
    log_length: usize,  
    last_log_term: TermId,  
}
```

No [Stateright](#), que foi onde testámos a falha, conseguimos facilmente encontrar um caso de teste que viola a propriedade de **Election Safety**.

Ao começar uma eleição, o nó malicioso envia mensagens de votação de todos os nós para ele próprio, de forma a que ele seja eleito, sem quaisquer verificações de outros nós. Desta forma, é possível eleger dois líderes diferentes num mesmo termo, o que viola a propriedade de **Election Safety**.

Localização: src/fault/message_forgery.rs

Ambiente de Teste

• Atores:

- Id(0) — **Malicioso**
- Id(1) — **Seguro**
- Id(2) — **Seguro**

Sequência de Eventos

1. Timeout(Id(1), ElectionTimeout)
2. Id(1) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(1), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(2)
3. Id(2) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(2), current_term: 1, vote_granted: true })) → Id(1)
4. Id(0) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(1), current_term: 1, vote_granted: true })) → Id(0)
5. Timeout(Id(0), ElectionTimeout)
6. Id(0) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(1), current_term: 1, vote_granted: true })) → Id(0)

Quando o nó malicioso (0) inicia uma eleição, ele simula que recebeu um voto do nó 1 (basta um para quórum), e assim é eleito como líder, mesmo que já exista outro líder no mesmo termo.

Estado Inválido Encontrado

```
Node {  
  id: Id(0),  
  current_role: Leader,  
  ...  
}
```

```
Node {  
  id: Id(1),  
  current_role: Leader,  
  ...  
}
```

```
Node {  
  id: Id(2),  
  current_role: Follower,  
  ...  
}
```

Propriedades Violadas

- [Segurança da Eleição \(Election Safety\)](#)

Medidas de Mitigação

Seria possível mitigar completamente este problema adicionando autenticação às mensagens, por exemplo, através de assinaturas digitais.

Demonstração das Medidas de Mitigação

Devido à natureza do problema, que não é relacionado com os conteúdos abordados na unidade curricular, não iremos implementar a mitigação.

Falta B - Voto Duplo

Identificação da Falta

Um nó malicioso pode votar em dois candidatos diferentes numa eleição no mesmo termo.

Impacto

Permite que dois nós sejam eleitos como líder, o que quebra a propriedade de **Election Safety** do algoritmo.

Demonstração do Impacto

Com o [Stateright](#), podemos injetar código na receção do `VoteRequest` e responder sempre positivamente ao voto, sem verificar se o nó já votou noutro candidato. Mais concretamente, o nó malicioso enviará sempre a mensagem `VoteResponse` com `vote_granted` a `true`, independentemente do pedido de voto que receber.

Localização: `src/fault/double_vote.rs`

Ambiente de Teste

• Atores:

- `Id(0)` — **Malicioso**
- `Id(1)` — **Seguro**
- `Id(2)` — **Seguro**

Sequência de Eventos

1. `Timeout(Id(0), ElectionTimeout)`
2. `Timeout(Id(2), ElectionTimeout)`
3. `Id(0) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(0), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(1)`
4. `Id(0) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(0), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(2)`
5. `Id(2) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(2), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(0)`
6. `Id(0) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(0), current_term: 1, vote_granted: true })) → Id(2)`
7. `Id(1) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(1), current_term: 1, vote_granted: true })) → Id(0)`

Podemos observar que o nó malicioso começa a atuar no evento n.º6, que deveria de responder ao voto do nó 2, como `vote_granted: false`, já que tem uma eleição em curso com o `log_length` maior ou igual ao do nó 2, mas responde `vote_granted: true`. Com o nó 1 a também responder `vote_granted: true` ao nó 0, o nó 0 também é eleito como líder.

Estado Inválido Encontrado

```
Node {  
  id: Id(0),  
  current_role: Leader,  
  ...  
}
```

```
Node {  
  id: Id(1),  
  current_role: Follower,  
  ...  
}
```

```
Node {  
  id: Id(2),  
  current_role: Leader,  
  ...  
}
```

Propriedades Violadas

- [Segurança da Eleição \(Election Safety\)](#)

Medidas de Mitigação

É possível resolver este problema com uma mensagem adicional, por exemplo, `ElectedBy`, que é difundida no fim da eleição por todos os nós eleitos, possuindo esta a lista de nós votantes. Caso seja detetado um nó que votou em dois candidatos diferentes, outros nós podem ignorar o seu voto futuramente (*black-listed*).

Daqui surge um novo problema, que é, um nó bizantino pode dizer que recebeu votos de quem não votou nele, o que faria que esse outro nó fosse *black-listed* indevidamente. Uma solução possível a este problema é recorrer a assinaturas digitais, que permitiriam identificar a autenticidade do voto.

Demonstração das Medidas de Mitigação

Como a segunda parte desta solução não é abordada na unidade curricular, não iremos implementar a mitigação com assinaturas digitais.

No entanto, a primeira parte da solução pode ser implementada, e é o que faremos.

Localização: `src/fault/double_vote_fix.rs`

Ambiente de Teste

- **Atores:**
 - `Id(0)` — **Malicioso**
 - `Id(1)` — Seguro com mitigação
 - `Id(2)` — Seguro com mitigação
- **Propriedades a verificar:**
 - O nó malicioso é **black-listed**
 - Expectativa temporal: `Sometimes`

Não conseguimos garantir todas as propriedades de segurança imediatamente após a eleição, porque o nó malicioso ainda pode causar a existência de dois líderes ao mesmo tempo.

Isto acontece porque a mensagem `ElectedBy`, que permite detetar votos duplicados, só é enviada no final da eleição. Assim, até que essa mensagem seja processada e o nó malicioso seja identificado e colocado na lista negra (*black-listed*), ainda pode haver uma violação temporária da propriedade de “Election Safety”.

No entanto, após o nó malicioso ser *black-listed*, é invocada uma eleição imediatamente, e os seus votos passam a ser ignorados nas eleições seguintes, restaurando o funcionamento correto do algoritmo e prevenindo futuras violações desta propriedade.

Apenas precisamos de garantir que o nó malicioso é colocado na lista negra (*black-listed*). As restantes propriedades já foram validadas em cenários sem nós maliciosos, e, após o *black-list*, o sistema volta a funcionar como se não existissem nós maliciosos.

Sequência de Eventos

1. `Timeout(Id(2), ElectionTimeout)`
2. `Id(2) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(2), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(0)`

3. Timeout(Id(1), ElectionTimeout)
4. Id(1) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(1), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(0)
5. Id(2) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(2), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(1)
6. Id(0) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(0), current_term: 1, vote_granted: true })) → Id(1)
7. Id(0) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(0), current_term: 1, vote_granted: true })) → Id(2)
8. Id(1) → Other(ElectedBy { leader: Id(1), term: 1, by: [Id(1), Id(0)] }) → Id(1)
9. Id(2) → Other(ElectedBy { leader: Id(2), term: 1, by: [Id(2), Id(0)] }) → Id(1)

O nó malicioso (0) viola o protocolo ao conceder voto positivo a dois candidatos diferentes (1 e 2) durante o mesmo termo de eleição. Como resultado, os nós 1 e 2 acreditam ter obtido a maioria dos votos e assumem simultaneamente o papel de líder, o que viola a propriedade de “Election Safety” do algoritmo de *Raft*. Esta situação só é detetada após o envio das mensagens *ElectedBy*, quando os nós honestos percebem que o mesmo nó (0) votou em mais do que um candidato. A partir desse momento, o nó malicioso é colocado na lista negra (*black-listed*) e os seus votos deixam de ser considerados em eleições futuras, restaurando a segurança do sistema.

Neste caso, apenas o nó 1 detetou inicialmente o problema; no entanto, em eventos subsequentes, o nó 2 também irá identificar a infração.

Estado Inválido Encontrado

```
Node {
  id: Id(0),
  blacklist: [],
  current_role: Follower,
  ...
}
```

```
Node {
  id: Id(1),
  blacklist: [Id(0)],
  current_role: Follower,
  ...
}
```

```
Node {
  id: Id(2),
  blacklist: [],
  current_role: Leader,
  ...
}
```

Propriedades Verificadas

- Nó Malicioso é **black-listed**

Falta C - Negação de Serviço por Spam de Eleições

Identificação da Falta

Cada *follower* tem um *timer* para dar *timeout* e começar uma eleição, quando não recebe atualizações de um líder. Um nó malicioso pode simplesmente ignorar esse *timer* e começar sempre uma nova eleição. A cada momento desses, o nó malicioso, incrementa o seu *term* e tenta eleger-se como líder. Como o seu *term* é maior que o do líder, todos os nós (incluindo o líder) votam nele, e ele passa a ser o líder.

Este nó malicioso pode continuar a fazer isto indefinidamente, sempre que um líder é eleito.

Impacto

Como não há um líder estável, a propriedade de *liveness* não é respeitada, isto é, o sistema não consegue fazer progresso, visto que não há tempo suficiente para replicar as entradas no *log*.

Demonstração do Impacto

Com o [Stateright](#), podemos fazer com que a cada mensagem recebida o nó malicioso comece uma eleição.

Localização: `src/fault/election_spam.rs`

Ambiente de Teste

- **Atores:**

- Id(0) — **Malicioso**
- Id(1) — **Malicioso**
- Id(2) — **Seguro**

Inicialmente, não tínhamos considerado que, com apenas um nó malicioso, o [Stateright](#) poderia encontrar cenários em que todas as propriedades de segurança fossem satisfeitas. Isto acontece porque, se o nó malicioso atrasar o envio das mensagens, os outros dois nós ainda conseguem formar uma maioria e executar o algoritmo de *Raft* corretamente.

Por isso, foi necessário ajustar o teste para incluir dois nós maliciosos. Assim, garantimos que há sempre eleições a decorrer, impedindo a verificação da propriedade de *Log Liveness*.

Ainda assim, mesmo com dois nós maliciosos, o [Stateright](#) encontrou uma possibilidade de ainda haver líder na implementação do *spam*. Se um dos nós maliciosos começar uma eleição e o nó 2 responder positivamente, o nó que começou a eleição passa a ser o líder.

Propriedades Não Verificadas

- [Vivacidade do Log \(Log Liveness\)](#)

Medidas de Mitigação

Para este problema, não há uma solução concreta, e teremos que recorrer a soluções heurísticas.

Uma solução possível é ignorar começos de eleições demasiado cedo, por exemplo, se um *term* começou há menos de 1 minuto, não vamos participar em eleições futuras. No entanto, esta solução pode fazer com que o sistema demore demasiado tempo a voltar ao normal caso o líder realmente morra nesse intervalo de tempo.

Outra solução possível é limitar o número de eleições que um nó pode iniciar num dado espaço de tempo, por exemplo, 1 eleição por hora.

Demonstração das Medidas de Mitigação

Optámos por uma abordagem mais simples: se um nó iniciar eleições em três ocasiões consecutivas, esse nó é colocado na lista negra (*black-listed*). Desta forma, previne-se que nós maliciosos possam continuamente desestabilizar o sistema através do *spam* de eleições.

Localização: src/fault/election_spam_fix.rs

Ambiente de Teste

- **Atores:**

- Id(0) — **Malicioso**
- Id(1) — Seguro com mitigação
- Id(2) — Seguro com mitigação

- **Propriedades a verificar:**

- [Segurança da Eleição \(Election Safety\)](#)
- [Vivacidade do Log \(Log Liveness\)](#)
- [Vivacidade de Eleições \(Election Liveness\)](#)
- O nó malicioso é **black-listed**
 - Expectativa temporal: Sometimes

Todas as propriedades de segurança foram verificadas com sucesso. No entanto, apenas demonstraremos a sequência de eventos que levaram à verificação do nó malicioso estar na lista negra (*black-listed*).

Sequência de Eventos

1. Timeout(Id(0), ElectionTimeout)
2. Id(0) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(0), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(2)
3. Id(2) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(2), current_term: 1, vote_granted: true })) → Id(0)
4. Id(0) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(0), current_term: 2, log_length: 0, last_log_term: 0 })) → Id(2)
5. Id(2) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(2), current_term: 2, vote_granted: true })) → Id(0)
6. Id(0) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(0), current_term: 3, log_length: 0, last_log_term: 0 })) → Id(2)

Neste cenário, o nó malicioso (0) inicia três eleições consecutivas, sendo que o nó 2 recebe todos esses pedidos. Após a terceira tentativa, o nó 0 é colocado na lista negra (*black-listed*) do nó 2, impedindo-o de continuar a desestabilizar o sistema.

Estado Válido Encontrado

```
Node {  
  id: Id(0),  
  blacklist: [],  
  current_role:
```

```
Node {  
  id: Id(1),  
  blacklist: [],  
  current_role: Follower,
```

```
Node {  
  id: Id(2),  
  blacklist: [Id(0)],  
  current_role: Follower,
```

```
Candidate,  
    ...  
}
```

```
    ...  
}
```

```
    ...  
}
```

O resto das propriedades podem ser verificadas rodando o teste unitário que executa o [Stateright](#), presente em `src/fault/election_spam_fix.rs`.

Propriedades Verificadas

- Nó Malicioso é **black-listed**
- [Segurança da Eleição \(Election Safety\)](#)
- [Vivacidade de Eleições \(Election Liveness\)](#)
- [Vivacidade do Log \(Log Liveness\)](#)

Falta D - Bifurcação de Log

Identificação da Falta

Um nó malicioso pode enviar, no mesmo índice e termo, duas versões diferentes do *log* para *subsets* distintos de nós. Com isto, diferentes *quorum's* de nós recebem versões diferentes do *log*, o que faz proporcionar uma bifurcação do *log*.

Impacto

Se dois conjuntos de nós aplicarem entradas diferentes no mesmo índice, a propriedade de **Log Safety** é violada: leituras ou escritas podem comportar-se de forma inconsistente em caso de falhas do líder.

Demonstração do Impacto

Para demonstrar essa vulnerabilidade, podemos considerar um cenário em que um nó malicioso, a cada LogRequest, incrementa todos os valores das entradas do *log* em uma unidade antes de enviá-las. Esse comportamento resulta em divergência entre as entradas *committed* dos diferentes nós, comprometendo a consistência do sistema.

Localização: src/fault/forking_log.rs

Ambiente de Teste

• Atores:

- Id(0) — **Malicioso**
- Id(1) — **Seguro**
- Id(2) — **Seguro**

Sequência de Eventos

1. Timeout(Id(1), ElectionTimeout)
2. Id(1) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(1), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(0)
3. Id(0) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(0), current_term: 1, vote_granted: true })) → Id(1)
4. Id(1) → Raft(LogRequest(LogRequestMessage { leader_id: Id(1), current_term: 1, prefix_length: 0, prefix_term: 0, commit_length: 0, suffix: [] })) → Id(0)
5. Id(0) → Raft(LogResponse(LogResponseMessage { node_id: Id(0), current_term: 1, ack: 0, success: true })) → Id(1)
6. Id(1) → ClientRequest(Write { key: 1, value: 42, msg_id: 1 }) → Id(1)
7. Id(1) → Raft(LogRequest(LogRequestMessage { leader_id: Id(1), current_term: 1, prefix_length: 0, prefix_term: 0, commit_length: 0, suffix: [LogEntry { term: 1, from: Id(1), request: Write { key: 1, value: 42, msg_id: 1 } }] })) → Id(0)
8. Id(0) → Raft(LogResponse(LogResponseMessage { node_id: Id(0), current_term: 1, ack: 1, success: true })) → Id(1)

Observa-se que, no evento n.º0, o nó 1 é eleito como líder. Já no evento n.º7, o nó 0 — que é malicioso — recebe a entrada do log, incrementa o valor e, assim, introduz uma inconsistência no log. Posteriormente, no próximo evento, ao receber a confirmação do nó 0, o nó 1 realiza o *commit* dessa entrada adulterada, consolidando a inconsistência no sistema.

Estado Inválido Encontrado

```
Node {
  id: Id(0),
  log: [
    LogEntry {
      term: 1,
      from: Id(1),
      request: Write {
        key: 1,
        value: 43,
        msg_id: 1,
      },
    },
  ],
  commit_length: 0,
  current_role: Follower,
  ...
}
```

```
Node {
  id: Id(1),
  log: [
    LogEntry {
      term: 1,
      from: Id(1),
      request: Write {
        key: 1,
        value: 42,
        msg_id: 1,
      },
    },
  ],
  commit_length: 1,
  current_role: Leader,
  ...
}
```

```
Node {
  id: Id(2),
  log: [],
  commit_length: 0,
  current_role: Follower,
  ...
}
```

Propriedades Não Verificadas

- [Segurança do Log \(Log Safety\)](#)

Medidas de Mitigação

Uma possível solução para esta falha, seria aquando a receção de um LogRequest contendo novas entradas do *log*, enviar para todos os outros nós, uma mensagem adicional que contivesse um *hash* calculado sobre o *log* atual (até à posição de *commit*). Desta forma, ao receber a mensagem, os outros nós fariam *hash* do seu *log* até à posição de *commit* que receberam (ou até ao seu próprio índice de *commit* caso seja maior) e comparariam os dois *hashes*. Caso fossem diferentes, o líder seria bloqueado pelos vários nós (*black_listed*) que detetassem a divergência, e começariam uma nova eleição.

No entanto, para além desta solução ser custosa em termos de tráfego de mensagens, não haveria forma de reestabelecer o *log*, já que não saberemos qual seria o *log* real.

Para além disso, faria com que existisse outro problema, um nó malicioso poderia agora simplesmente criar *hashes* falsos, podendo bloquear qualquer líder.

Desta forma, decidimos não implementar uma solução para este problema.

Falta E - Commit de Log sem ser Aceite pela Maioria

Identificação da Falta

Um nó líder malicioso pode incrementar o `commit_index` sem ter recebido confirmação da maioria dos nós.

Impacto

Os *logs* podem não estar atualizados numa maioria de nós, o que faria com que, caso o líder falhasse, dados fossem perdidos. Isto quebra o princípio de *safety*, pois o sistema pode considerar como *committed* entradas que, na realidade, não foram replicadas na maioria dos nós.

Por exemplo, imaginando que um líder recebe um pedido de escrita e adiciona a entrada ao seu *log*, mas, devido a uma falha de comunicação, apenas um dos seguidores recebe essa entrada, enquanto o outro não recebe nada. Se o líder for malicioso e incrementar o `commit_index` mesmo sem ter confirmação da maioria, ele pode responder ao cliente como se a operação estivesse garantida. Se este líder falhar imediatamente a seguir, e um novo líder for eleito entre os nós que não têm a entrada no *log*, a operação será perdida para sempre, apesar de já ter sido considerada *committed* pelo sistema.

Demonstração do Impacto

Com o [Stateright](#), podemos criar um nó malicioso que incrementa o `commit_index` para o tamanho do *log* depois de qualquer mensagem ter sido enviada.

Localização: `src/fault/fake_commit_log.rs`

- **Atores:**

- Id(0) — **Malicioso**
- Id(1) — Seguro
- Id(2) — Seguro
- Id(3) — Seguro
- Id(4) — Seguro

- Número de *crashes* possíveis: 1

- Mensagens possíveis:

- `ClientRequest::Write { key: 1, value: 42, msg_id: 1 }`
- `ClientRequest::Write { key: 2, value: 43, msg_id: 2 }`

Apesar de termos configurado um ambiente de teste com 5 nós, permitindo mortes (*crashes*) e utilizando duas mensagens distintas para criar divergência nas entradas do *log*, o [Stateright](#) não detetou qualquer violação das propriedades. Não é claro se isto se deve a limitações na profundidade de exploração do [Stateright](#), à inexistência efetiva do problema neste cenário, ou à impossibilidade de simular partições de rede entre os atores, uma vez que o [Stateright](#) não suporta este tipo de falha. Teríamos interesse em investigar mais a fundo as causas desta ausência de falhas detetadas, mas, devido a limitações de tempo, não nos foi possível fazê-lo.

Medidas de Mitigação

Este problema poderia ser resolvido, enviando uma assinatura digital como forma de autenticidade em cada `LogResponse`. Assim, o líder era obrigado a guardar essas assinaturas, para as propagar a seguir. Os outros nós só aceitariam `LogRequest`'s que incrementem o `commit_index`, se estes apresentassem assinaturas digitais válidas de um *quorum* de nós.

Demonstração das Medidas de Mitigação

Da mesma forma que na solução da Falta A, a natureza do problema não está relacionada com os conteúdos abordados na unidade curricular, portanto não iremos implementar a mitigação.

Conclusão

Este trabalho prático permitiu ao grupo aprofundar significativamente o conhecimento sobre o algoritmo de *Raft*, as suas complexidades e as implicações de faltas assertivas na sua operação. A exploração de diferentes cenários de falhas e investigação de mitigações eficazes proporcionaram uma perspetiva valiosa sobre o delicado balanço entre garantir a correção do sistema e manter um bom desempenho.

A utilização do [Stateright](#) revelou-se uma excelente escolha, visto oferecer uma experiência de utilização consideravelmente superior à do Maelstrom para a verificação formal de propriedades, identificação de vulnerabilidades e exploração de possíveis estados.

Adicionalmente, o grupo gostaria de ter tido a oportunidade de expandir a funcionalidade da aplicação principal. Seria interessante que, para além da integração com o Maelstrom, fosse possível executar nós *Raft* com as faltas implementadas ativadas diretamente, dado que a “API de Faltas” foi concebida de forma genérica sobre a implementação do *Raft*. Outra área de exploração futura seria a integração mais dinâmica com o [Stateright](#), permitindo, por exemplo, a execução da sua interface gráfica (UI) com atores e cenários definidos dinamicamente, facilitando uma análise ainda mais interativa e detalhada do comportamento do sistema.



Figura 1: Interface gráfica do [Stateright](#)